Лабораторно упражнение 8

Структурни шаблони. Шаблон „Адаптер“ (Adapter)

I. Теоретична част

1. Шаблон „Адаптер“ - същност

1.1. Цел и мотивация

Цел: Да преобразува интерфейса на един клас в интерфейс на друг клас, какъвто очакват клиентите. Прави се възможна работата на класове с несъвместими интерфейси.

Мотивация:

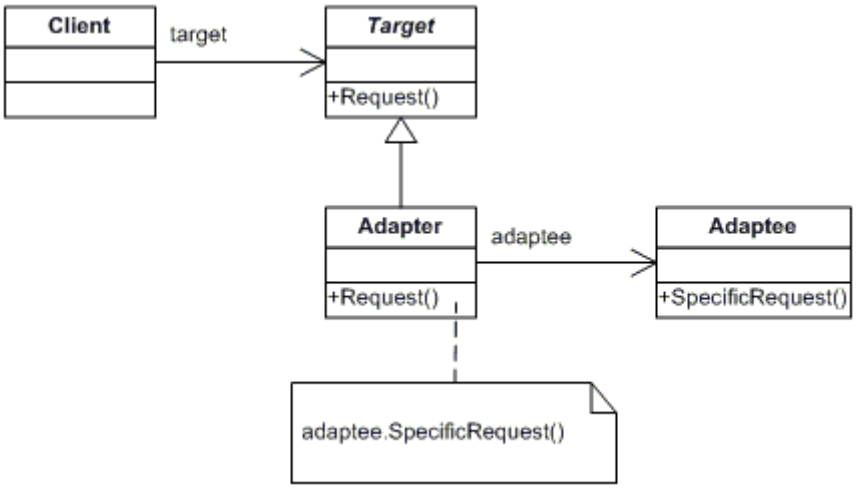
Не всички системи имат добър дизайн. Дори и ако модулът или приложението първоначално са били добре обмислени, внясятето на промени от различни разработчици с течение на времето би могло да доведе до неприятни последствия – несъгласувани реализации на еднотипни задачи.

1.2. UML диаграма

Диаграмата на шаблон „Адаптер“ е представена на фигура 1.

В диаграмата участват следните класове и обекти:

- Target – дефинира целевия интерфейс, който клиентът използва и към който трябва да бъдат преобразувани интерфейсите на вече съществващите класове
- Adaptee – дефинира съществуващият интерфейс, който трябва да бъде адаптиран
- Adapter – клас адаптер, който преобразува интерфейса Adaptee към интерфейса Target
- Client – осъществява връзката между обектите, приспособени чрез интерфейса Target; клиент на новия интерфейс, който работи с адаптираните класове по полиморфен начин.



Фиг. 1. UML диаграна на шаблон „Адаптер“

1.3. Програмна реализация

Представената на фиг. 3 UML диаграма съответства на следния примерен структурен програмен код, представящ шаблона „Адаптер“:

```
using System;
namespace Adapter
{
    class Program
    {
        static void Main(string[] args)
        {
            // клиентски код
            Target target = new Adapter();
            target.Request();
        }
    }
    // интерфейс, до който трябва да бъде адаптиран класът
    public class Target
    {
        public virtual void Request()
        {
            Console.WriteLine("Called Target Request()");
        }
    }

    // клас адаптер
    public class Adapter : Target
    {
        private Adaptee adaptee = new Adaptee();
        public override void Request()
        {
            // извикване на метода на класа, който трябва да бъде адаптиран
            adaptee.SpecificRequest();
        }
    }
    // клас, който трябва да бъде адаптиран
    public class Adaptee
    {
```

```
        public void SpecificRequest()
        {
            Console.WriteLine("Called SpecificRequest()");
        }
    }
}
```

В случая е класът `Adaptee`, който трябва да бъде адаптиран. Достъпът до неговите данни е чрез метода `SpecificRequest`. Клас `Target` играе роля на интерфейса, до който трябва да бъде адаптиран класът `Adaptee`. Клас `Adapter` наследява клас `Target` и извиква метод `SpecificRequest`. В клиентския код се работи с адаптирания клас, но се достъпват данните на класа, който трябва да бъде адаптиран.

II. Задачи за изпълнение

1. Резултатите от изпит на даден студент се получават от изпитната платформа като масив от низове в следната последователност: име, фамилия, факултетен номер, e-mail адрес, почлучен брой точки. Пример е клас `Group`:

```
using System;
using System.Collections.Generic;
using System.Text;

namespace StudentsAdapter
{
    // class Adaptee
    class Group
    {

        private string [] data = { "Иван", "Иванов", "21905111",
"i.ivanov@gmail.com", "18,5" };

    public string GetItem(string item)
        {
            switch(item.ToLower())
            {
                case "name": return data[0];
                case "family":return data[1];
                case "id":return data[2];
                case "email":return data[3];
                case "score": return data[4];
                default: return "";
            }
        }
    }
}
```

Клас `Group` съдържа метод `GetItem` , който връща стойността на съответния елемент.

Като се използва шаблон адаптер да се създадат следните класове: абстрактен клас Student (Target class), който описва студент със слените данни: име, фамилия, факултетен номер, e-mail адрес, получен брой точки. В класа да бъдат включени методи за: задаване стойности на данните; пресмятане на получената оценка; показване резултат от изпит; генериране на данни

за протокол. Класът Student се наследява от клас MyStudent, който имплементира методите на абстрактния клас.
Оценката се получава по следния начин: от 13,5 до 17,5 точки – Среден (3); от 18 до 22 т – Добър (4); от 22,5 до 25 – Мн. добър (5); от 25,5 и нагоре – Отличен (6).
Данните за резултатите на студента да включват: имена, фак. Номер, получен брой точки, оценка.
Данните за протокол да включват: имена, фак. номер, оценка.
Да се създаде приложение, което демонстрира работата на шаблона Адаптер.